Tabla de contenidos

1. Git
2. Instalación
3. Entendiendo Git
4. Comandos básicos
5. Entendiendo un commit
6. Token de acceso personal

Git

Git es un sistema distribuido de **control de versiones** ampliamente utilizado en el mundo del desarrollo de software. En esta ocación, lo necesitarás para poder descargar los contenidos y enunciados del curso, y para entregar todas tus evaluaciones a lo largo del curso. A continuación, podrás ver guías de cómo instalarlo, y presentaremos los comandos básicos que necesitarás para poder utilizar git correctamente a lo largo del curso.

Instalación

El proceso de instalación dependerá de tu sistema operativo. Antes de lanzarte a instalar git, puedes corroborar si es que ya lo tienes instalado con el siguiente comando en consola:

```
git --version
```

Si es que te muestra un número de versión, quiere decir que ya lo tienes instalado y no debes seguir los pasos de instalación. En caso de que no te reconozca el comando, quiere decir que debes instalar git, siguiendo las instrucciones de tu sistema operativo:

Windows

Puedes descargar el instalador directamente desde la página de Git aquí.

Linux

Es probable que ya esté instalado, pero de no ser así lo puedes instalar con:

- Distribuciones Debian-Like:

```
sudo apt install git
```

- Distribuciones Arch-Like:

```
sudo pacman -S git
```

macOS

Para usar git en macOS, es necesario tener las herramientas de desarrollo de línea de comando de Xcode (conocidas como *xcode command-line developer tools* en inglés). Estas entregan un conjunto de herramientas para desarrollar en tu macOS.

Para instalarlas, solo debes ejecutar

```
xcode-select --install
```

en tu consola.

Si no tienes las herramientas instaladas, aparecerá un mensaje que te dará la opción de instalar las herramientas (*Install*) o de además instalar Xcode (*Get Xcode*). Para desarrollar, solo necesitas las herramientas, por lo que te recomendamos no instalar Xcode, pues es una aplicación muy grande y probablemente tome tiempo en descargarla.

Por el contrario, si ya las tienes instaladas, el comando debería mostrar el mensaje `xcode-select: error: command line tools are already installed, use "Software Update" to install updates`.

También necesitarás de Homebrew para instalar git. Puedes verificar si lo tienes mediante el siguiente comando

```
brew -v
```

Si no tienes Homebrew instalado, lo puedes hacer ejecutando el siguiente comando en el terminal

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

puedes encontrar más información sobre Homebrew aquí. Ahora, ya estás con todo lo necesario para comenzar a utilizar git.

Una vez que tengas las herramientas, puedes proceder a actualizar git. Este viene por default en macOs, pero si ejecutas

```
brew install git
```

en la consola, se actualizará a la última versión. También, puedes encontrar el instalador aquí.

Entendiendo Git

Si bien Git provee muchas herramientas, en el curso solo será necesario entender lo básico para poder desenvolverse de manera correcta en el curso. Para eso, necesitamos entender qué es un repositorio en git, y cómo se maneja.

En términos prácticos, un repositorio es un almacenamiento virtual de un conjunto de archivo, y almacena no solo el estado final de los archivos que contiene, sino también almacena todas las

versiones que se han ido almacenando en él. Al utilizar git, manejaremos 4 ubicaciones de los archivos: la carpeta del computador, la lista de cambios, el repositorio local y el repositorio remoto.

Comandos básicos

Para utilizar git en el curso, solo necesitarás 5 comandos a lo largo de todo el semestre: `clone`, `pull`, `add`, `commit` y `push`. Todos los comandos de git puedes correrlos desde la consola, por lo que si tienes alguna duda por ese lado puedes volver a leer el material de cómo usar una terminal.

git clone

Con este comando podrás clonar tu repositorio, el repositorio de los contenidos y el syllabus en tu computador. Para esto, puedes abrir una terminal en la carpeta en donde desees guardar los repositorios, y ejecutar los siguientes comandos:

```
git clone https://github.com/IIC2233/Syllabus.git
git clone https://github.com/IIC2233/contenidos.git
git clone https://github.com/IIC2233/<tu_usuario_git>-iic2233-2025-1.git
```

Si es que el clonado fue exitoso, entonces deberías ver en tu computador las 3 carpetas asociadas a cada uno de los repositorios.

git pull

Como se mencionó anteriormente, está el repositorio remoto y el repositorio local. Si es que hay archivos o cambios que están en el repositorio remoto pero no están en el repositorio local de tu computador, para traerte los cambios debes hacer

```
git pull
```

Este comando lo necesitarás durante el curso para descargar los enunciados de las tareas, descargar los contenidos que se vayan subiendo y actualizar tu repositorio en caso de que utilices más de un computador para subir cambios.

git add

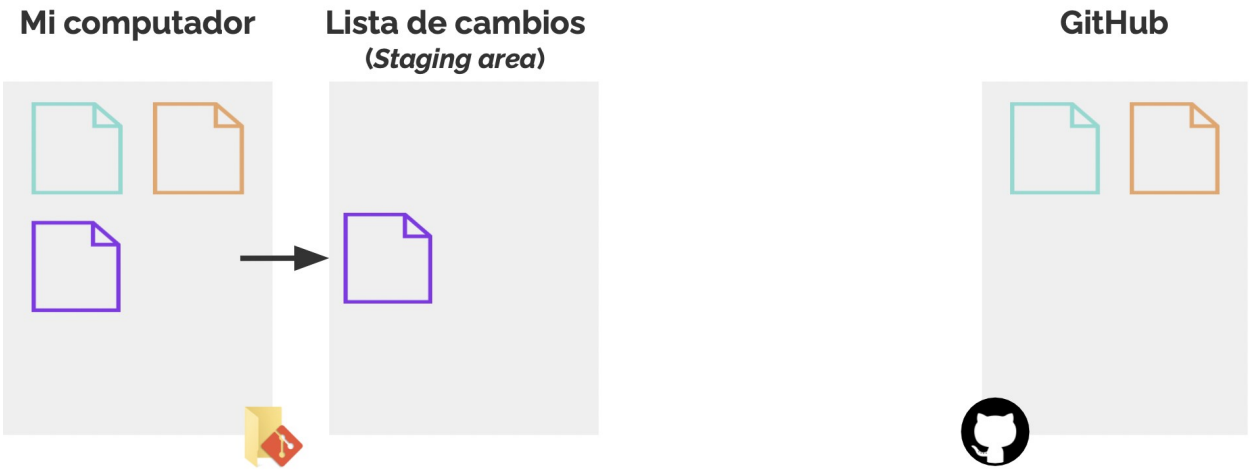
Para entender el flujo de subida, vamos a ir ejemplificando con el siguiente escenario: Se tiene un archivo en el computador (archivo morado) llamado README.md, que uno quiere hacer que llegue al repositorio remoto. Para esto, tendremos que usar distintos comando de git para hacer que funcione.



En una primera instancia, debemos informarle a git que tenemos la intención de agregar el archivo a nuestro repositorio. Para esto, podemos hacer

```
git add README.md
```

El comando recibe el nombre (o el path) del archivo que queremos agregar al repositorio. Una vez hecho esto, nuestro archivo pasa a estar en la lista de cambios, pero aún no está en nuestro repositorio. En la lista de cambios podemos poner tantos archivos como queramos, por lo que podemos ir agregando varios de una misma vez, para luego agregarlos al repositorio todos juntos. Una vez hecho el `git add README.md`, el estado quedaría así:



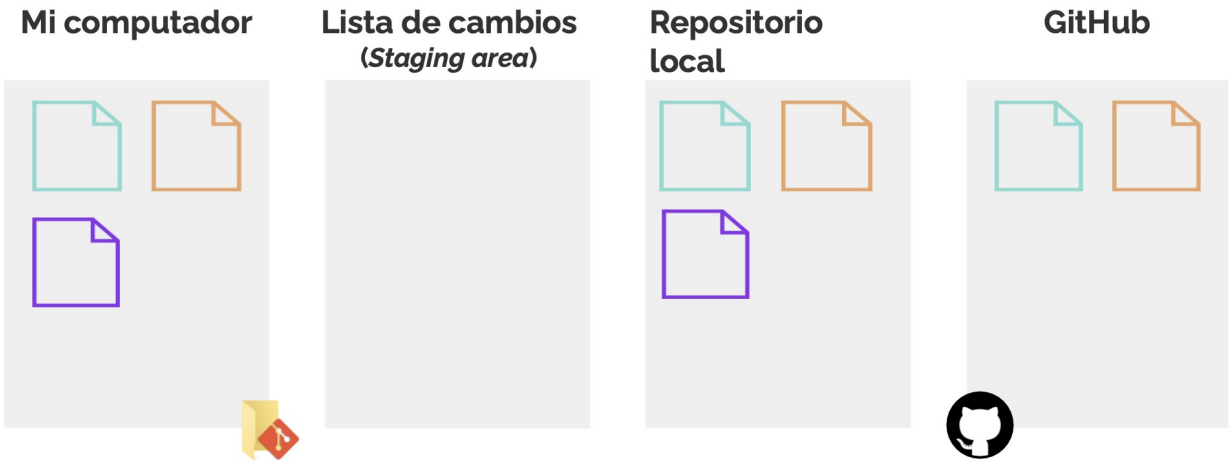
Importante notar, que todavía no se ha añadido el archivo ni los cambios al repositorio, por lo que aún no se ha generado una versión de nuestros archivos dentro del repositorio.

git commit

Como mencionamos anteriormente, git es un sistema de control de versiones. Para crear una versión de nuestros archivos, debemos usar la instrucción `commit`. Para crear una versión, podemos ejecutar el comando

```
git commit -m 'mensaje descriptivo explicando lo que estoy agregando al repositorio'
```

Una vez hecho esto, se crea una versión de nuestros archivos, y se agregan todos los cambios y archivos que estaban en la lista de cambios a una nueva versión dentro del repositorio. Estas versiones guardan una serie de datos (que muchas no son tan relevantes para el curso), pero lo importante es que queda un registro de la hora en la que se hizo el commit, y los cambios y archivos que se incluyeron con dicho commit. Habiendo hecho esta instrucción, nuestro estado ahora es el siguiente:



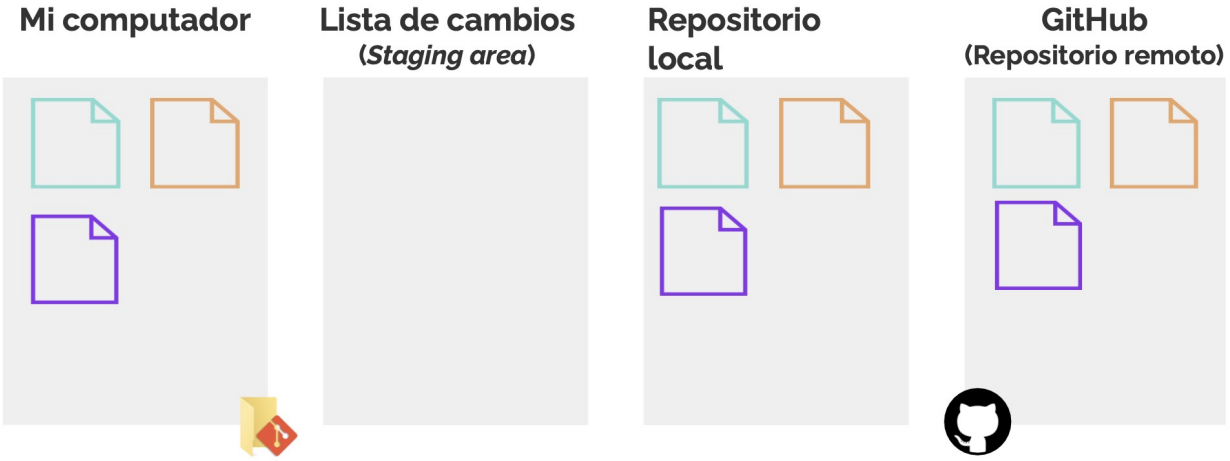
El archivo morado ya es parte del repositorio local, pero aún no está en el repositorio remoto.

git push

La última instrucción que nos servirá para este curso, es el `push`. Lo que hace esto, es publicar y subir al repositorio remoto todas las versiones (o commits) que estén en el repositorio local que aún no hayan sido subidas al repositorio remoto. Para hacer esto, basta con ejecutar

```
git push
```

y se subirán los cambios al repositorio remoto, y el estado ahora es el siguiente:



Entendiendo un commit

Ahora que ya sabes cómo usar los comandos básicos de Git, es importante entender **qué contiene realmente cada commit** y por qué esto es fundamental para el control de versiones.

Pregunta común de estudiantes

"Si subo el README.md al final, ¿al descargar el último commit solo obtengo el README?"

Respuesta: ¡NO! Cada commit contiene **todos** los archivos de commits anteriores.

Analogía de la Muñeca Rusa (Matryoshka)

Imagina que cada commit que haces es una muñeca rusa. Al igual que estas muñecas, los commits se guardan uno dentro del otro, en orden cronológico. La muñeca más grande representa el último commit, y dentro de ella están todas las anteriores.

Observa la primera imagen:

En esta representación:

1. El primer commit (muñeca más pequeña) solo tiene un archivo: main.py.
2. El segundo commit incluye ese archivo y agrega utils.py.
3. El tercer commit es más grande porque contiene los dos anteriores más un nuevo archivo: README.md.

Cada nueva muñeca encapsula todo lo que contenía la anterior, más los archivos agregados o modificados en ese momento. Así funciona Git: cada commit guarda el estado completo del proyecto.

El último commit no está solo

Se puede observar que el último commit no contiene solo el archivo README.md. De hecho, si haces un git clone, Git no te entrega solo lo último que escribiste. Te entrega todo el proyecto hasta el punto de ese commit, incluyendo todos los archivos previos, como main.py y utils.py.

Esta es una parte fundamental del control de versiones: Git permite volver atrás, comparar versiones, o recuperar cualquier archivo desde un commit anterior porque todos los commits son estados completos del proyecto, no fragmentos incompletos.

¿Por qué es importante entender esto?

Es común creer que al hacer un commit en Git es necesario adjuntar todos los archivos del proyecto, como si se tratara de una entrega en Canvas. En Canvas, cuando subes una tarea, debes asegurarte de incluir todos los archivos relevantes, porque si olvidas uno, la entrega queda incompleta. En cambio, Git no funciona así. Git no espera que tu proyecto esté terminado

al momento de hacer un commit.Por lo que, puedes hacer commits con una sola parte del proyecto, incluso si todavía está en desarrollo. Y lo más importante: el commit seguirá conteniendo todo el historial de archivos anteriores.

Este cambio de paradigma es importante: en Git no estás "entregando" el proyecto cada vez, sino **registrando un punto en su evolución**. Por eso, es recomendable hacer entregas parciales, aunque el código no esté completo. Esto te permite llevar un registro claro de tu avance, recuperar versiones anteriores, y trabajar de forma más segura.

Token de acceso personal

Los *tokens* de acceso personal son una alternativa al manejo de contraseñas para autentificar usuarios de Github, especialmente cuando se usa la terminal para interactuar con un repositorio personal. En el curso, existirán repositorios de acceso público, y otros de acceso privado. Para la administración de los repositorios privados se exigirá el uso de este *token* especial, el cual es personal e intransferible. Puedes encontrar la documentación completa en este link de Github Docs. Además, la Actividad 0 de este semestre tendrá un paso a paso para crear y probar el correcto uso de tu repositorio personal.

Creación del token

Para crearlo y administrarlo, primero debes acceder a este enlace https://github.com/settings/tokens/new e ingresar con tu cuenta de Github.

Luego, debes completar un formulario del siguiente modo:

1. Agregar una descripción en la sección de *Note*. Este será un texto para darle una descripción al *token* generado.

2. Definir una fecha de expiración. Se recomienda poner una posterior al cierre de semestre. Una vez pase la fecha definida, el *token* ya no servirá y deberás generar uno nuevo.

3. En la sección de *Select scopes*, haz *click* en cada cuadrado. Así haremos un *token* con todos los permisos posibles. Más adelante, en la carrera, aprenderás a generar un *token* con permisos más exclusivos.

Tras cumplir con los 3 pasos, debes hacer *click* en un botón que está al final del formulario que dice **Generate token**. La vista de tu navegador se actualizará y te aparecerá un texto que comienza con `"ghp_"`. Ese será tu *token* que deberás guardar y usar siempre en tu terminal para interactuar con tu repositorio privado (*clone, pull, push*). No pierdas este *token* porque solo aparecerá una vez al momento de generarlo. En caso de olvidarlo, deberás crear uno nuevo siguiendo todos los pasos anteriores.

Primer uso del token

Tu token será pedido por primera vez al clonar tu repositorio personal. Una vez realizado el proceso se te pedirá ingresar tu usuario y luego el *token* que acabamos de generar. Este **no se verá en tu terminal al momento de copiar**, pero es nor mal porque es por temas de seguridad. Luego presionas *enter* y se comenzará a clonar tu repositorio.

Problemas comunes de Git

Si al trabajar en tu repositorio personal durante el transcurso del semestre usas Git de forma adecuada, no deberías encontrarte con ningún error o problema que te impida mantener tu repositorio local y repositorio remoto sincronizados. Sin embargo, como curso hemos detectado dos problemas que ocurren de forma frecuente entre los estudiantes, los cuales registramos a continuación junto con su solución:

Repositorio Des-sincronizado

Como explicamos, tu repositorio se conforma a partir de una serie de cambios progresivos a traves del tiempo. Git maneja automáticamente el orden en que fueron realizados estos cambios cuando se trabaja en un solo computador*. Sin embargo, si empezamos a editar nuestro repositorio desde dos orígenes distintos sin hacer una sincronización previa, Git no puede resolver el orden de los *commits* al momento de ordenar la linea de tiempo de cambios y avisará del problema en la terminal, evitando que uno pueda hacer un push nuevo hasta solucionar el conflicto.

Al trabajar desde más de un computador o incluso, si por alguna razón editamos desde la página web de Github los archivos de nuestro repositorio (usualmente nos tentaremos a editar el archivo README desde la página web), probablemente nos encontraremos con el siguiente problema cuando tratemos de subir algún cambio desde nuestro computador:

Esto es porque Git no sabe cómo dejar el historial de cambios: ¿Pone primero los cambios del computador y luego los de la página, o viceversa?. Como dice la advertencia, una posible solución es intentar primero hacer `git pull` y luego `git push` de nuevo. Sin embargo, en casos donde ambas versiones del repositorio editaron el mismo archivo, aparecerá un nuevo error, llamado **merge conflict**:

Este es un problema más delicado todavía, ya que si 2 lugares editaron un mismo archivo, Git no sabe qué cambios en el archivo son los que deberían quedar como versión final y por ende, le pide al usuario que lo arregle. Es posible arreglar estos problemas desde nuestra IDE o terminal, sin embargo es un proceso **bastante complejo** y que tiene el riesgo de **arruinar tus archivos de forma permanente** en caso de hacerse mal, por lo que como curso **no recomendamos intentar solucionar este problema desde la terminal, o siguiendo tutoriales en internet**.

Subir archivos muy pesados a Github

El siguiente problema tiene que ver con la plataforma Github, en vez de ser un problema de Git. Github tiene una restricción donde prohibe de forma automática subir archivos que pesen mucho (más de 100MB), por temas de espacio. Tu repositorio no debería contener archivos singulares que pesen tanto, ya que en general archivos de código no pesan mucho, y si deseamos almacenar cosas como fotos o videos que pesen mucho, debemos usar otros medios de almacenamiento y no Github.

Un error común que ocurre es que hagamos `git add` y `git commit`, incluyendo un archivo que pese más del límite, lo que sí será permitido por Git. Sin embargo, al momento de hacer `git push`, veremos el siguiente problema:

Desde este punto en adelante, aunque quitemos el archivo del repositorio y hagamos `git add`, `git commit` y `git push` nuevamente, no podremos subir cosas desde el repositorio a la nube. Esto se debe a que, aunque hayamos removido el archivo, como Git es un sistema que maneja el historial de cambios, dentro de este historial sigue siendo almacenado el hecho de que alguna vez estuvo ese archivo ahí, bloqueando indefinidamente al repositorio local de subir cosas a Github.

Cómo solucionar problemas de Git rápidamente

Una solución rápida a los problemas mencionados arriba (y en general, a cualquier problema de Git cuya solución implique usar un comando de Git que no fue visto en el curso), que además asegura que tus archivos/avances de código no se pierdan, es la siguiente:

- En otra ubicación de tu computador distinta a la donde se encuentra tu repositorio personal, vuelve a clonar tu repositorio usando el comando `git clone`. Con esto, tendrás una nueva copia de lo que está actualmente en Github.
- Desde la copia original de tu repositorio, que es la que está presentando errores, copia todas las carpetas y archivos que quieras mantener y pégalos en la nueva copia.
- Estando ubicado en la nueva copia del repositorio, haz el `git add`, `git commit` y `git push` respectivo. Con esto, estamos diciendo que nos quedaremos con la línea de tiempo que está en Github, y al final de esta empezaremos a agregar cambios nuevos.
- Revisa en Github que se hayan subido todos los archivos que deseabas mantener en tu repositorio, y que no perdiste información. Una vez tengas certeza de que tienes todo respaldado en la nube, **elimina la copia antigua del repositorio** (la que generaba problemas).

Esto debería resolver la mayoría de los problemas de Git que encuentres. No es necesario que sepas todos estos pasos de memoria, pero sí recuerda que este notebook tiene un paso a paso si alguna vez llegas a encontrarte con estos problemas.

Aprendizaje

La mejor forma de prevenir problemas de git es **mantener un repositorio personal** que esté siempre sincronizado con el de Github. Para esto:

- Cada vez que termines de trabajar en tu PC, haz `git add`, `git commit` y `git push` de tu avance.
- No edites tus archivos desde la página web de Github.
- Si sabes que tu repositorio remoto tuvo algún cambio, trata de hacer `git pull` lo antes posible para que tu repositorio local también tenga esos cambios.
- En caso de cualquier problema, seguir el proceso anterior de hacer una nueva copia y subir las cosas desde ella, en vez de seguir tutoriales de internet que usen comandos que no conozcas.

- Haz avances frecuentes en tu repositorio local y súbelos al remoto, para que no entregues fuera de plazo debido a un error de Git.

Y ante cualquier duda, pregunta a la brevedad en el foro del curso, donde alguien con más experiencia en Git podrá ayudarte.

---

* Y siempre que se trabaje en una sola rama. Para efectos de este curso, puedes asumir esta segunda condición siempre, ya que el manejo de ramas se ve en cursos más avanzados de programación.